

Rail Tickets System — Detailed Level Design (DLD)

Version: 2.0 Created by: Kamil Błasiak

1) System Architecture Overview

Layered View

- **UI Layer (Presentation)**
 - `ConsoleUI` renders tickets and messages to stdout. Responsibilities: formatting domain objects for display, no business logic.
- **Logic Layer (Application / Domain)**
 - `Cashier` orchestrates user-facing actions: search, block, sell, return, and reporting. Applies validation and state transition rules.
 - `IClock` (abstraction) and `SystemClock` (implementation) provide current time and a human-readable timestamp string for reporting.
- **Data Layer (Persistence / Access)**
 - `RailwayConnectionDatabase` manages `Ticket` storage in-memory, lookup, search, and status updates.
- **Types (Shared Domain Model)**
 - Value types and enums in `Types.hpp`: `Passenger`, `Ticket`, `CoachType`, `TicketStatus`.

Runtime Interaction (Happy-path)

1. Cashier requests a search from DB → DB returns available tickets matching destination → UI prints.
 2. Cashier blocks a selected ticket → DB updates status → UI acknowledges.
 3. Cashier sells a previously blocked ticket to a passenger → DB updates → UI confirms.
 4. Cashier returns a sold ticket given a future travel date → `IClock` supplies `now()` → policy calculates penalty → DB marks returned → UI prints refund.
-

2) UML Class Diagram (attributes & methods)

```
classDiagram
    class ConsoleUI {
        <<utility>>
        +showTicket(t: Ticket): void
        +showMessage(msg: string): void
    }
```

```

class Employee {
    -id: string
    +Employee(id: string)
    +getId(): string
    ~virtual ~Employee()
}

class IClock {
    <<interface>>
    +now(): time_t
    +now_str(): string
}

class SystemClock {
    +now(): time_t
    +now_str(): string
}

class RailwayConnectionDatabase {
    -tickets: vector<Ticket>
    +addTicket(t: Ticket): void
    +getTicketById(id: int): Ticket*
    +search(dest: string): vector<Ticket>
    +updateTicketStatus(id: int, newStatus: TicketStatus): bool
}

class Cashier {
    -db: RailwayConnectionDatabase*
    -clock: IClock*
    -soldTickets: vector<int>
    -returnedTickets: vector<int>
    +Cashier(id: string, db: RailwayConnectionDatabase*, clk: IClock*)
    +searchTickets(dest: string): void
    +blockTicket(ticketId: int): bool
    +sellTicket(ticketId: int, p: Passenger): bool
    +returnTicket(ticketId: int, travelDate: time_t): bool
    +generateReport(): void
}

Employee <|-- Cashier
IClock <|.. SystemClock
Cashier --> RailwayConnectionDatabase
Cashier --> IClock
ConsoleUI ..> Ticket
RailwayConnectionDatabase ..> Ticket

```

Notes: - **Employee** is a base class for **Cashier**. - **ConsoleUI** is stateless and offers static rendering helpers. - The DB holds **Ticket** entities in-memory; IDs are unique.

3) State Diagram — Ticket Lifecycle

```
stateDiagram-v2
    [*] --> AVAILABLE
    AVAILABLE --> BLOCKED: cashier.blocks()
    BLOCKED --> SOLD: cashier.sells()
    BLOCKED --> AVAILABLE: timeout/cancel (future extension)
    SOLD --> RETURNED: cashier.returns(travelDate) / refund(policy)
    RETURNED --> [*]
```

Rules enforced in code: - Only **AVAILABLE** → **BLOCKED** is allowed when blocking. - Only **BLOCKED** → **SOLD** is allowed when selling. - Only **SOLD** → **RETURNED** is allowed when returning. - Returning requires `travelDate now()`.

4) Enumerations & Domain Types

```
// Ticket status
enum class TicketStatus { AVAILABLE, BLOCKED, SOLD, RETURNED };

// Coach types
enum class CoachType { SLEEPER, COMPARTMENT, SEATER };

struct Passenger { string name; string socialSecurityCode; };

struct Ticket {
    int id; string destination; string originStation; CoachType coachType;
    double cost; TicketStatus status;
};
```

Validation and transition rules apply over `TicketStatus`.

5) Validation Rules & Policies

5.1 Blocking

- **Preconditions:** Ticket exists, `status == AVAILABLE`.
- **State transition:** `AVAILABLE` → `BLOCKED`.
- **Postconditions:** DB reflects `BLOCKED`, UI confirmation output.

- **Failure cases:** Missing ticket, or status `AVAILABLE` → reject with message.

5.2 Selling

- **Preconditions:** Ticket exists, `status == BLOCKED`, passenger info present.
- **State transition:** `BLOCKED` → `SOLD`.
- **Postconditions:** DB reflects `SOLD`, cashier records `ticketId` in `soldTickets`.
- **Failure cases:** Status `BLOCKED` → reject with message.

5.3 Returning & Refund Policy

- **Preconditions:** Ticket exists, `status == SOLD`; `travelDate >= now()`.
- **State transition:** `SOLD` → `RETURNED`.
- **Penalty (by days before travel):**
 - 30 days → 1%
 - 15 days → 5%
 - 3 days → 10%
 - < 3 days → 30%
- **Refund:** `refund = cost * (1 - penaltyRate)` rounded as per output formatting rules (current code prints raw double). Future enhancement: format currency.
- **Postconditions:** DB reflects `RETURNED`, cashier records `ticketId` in `returnedTickets`, UI shows refund and penalty.
- **Failure cases:** Not `SOLD`, travel date in the past.

5.4 Searching

- Filters tickets where `destination == dest` and `status == AVAILABLE`.

6) Class & Method Specifications (DLD level)

6.1 ConsoleUI (UI Layer)

- **Responsibility:** I/O formatting to console.
- **Methods:**
 - `showTicket(const Ticket& t): void` — prints id, route, price, and textual status.
 - `showMessage(const std::string& msg): void` — prints informational or error messages.

6.2 Employee

- **Responsibility:** Base identity for staff objects.

- **Attributes:** `id: string`.
- **Methods:** `getId(): string`.

6.3 IClock / SystemClock

- **Responsibility:** Abstract wall-clock; concrete system time provider.
- **Methods:**
 - `now(): time_t` — current epoch seconds.
 - `now_str(): string` — formatted YYYY-MM-DD HH:MM:SS.

6.4 RailwayConnectionDatabase (Data Layer)

- **Responsibility:** In-memory repository for tickets.
- **Attributes:** `tickets: vector<Ticket>`.
- **Methods:**
 - `addTicket(const Ticket& t): void` — append.
 - `getTicketById(int id): Ticket*` — linear search, pointer or nullptr.
 - `search(const string& dest): vector<Ticket>` — filter by destination & AVAILABLE.
 - `updateTicketStatus(int id, TicketStatus newStatus): bool` — set status if found.

6.5 Cashier (Logic Layer)

- **Responsibility:** Orchestrate ticket operations and report generation.
- **Attributes:** `db, clock, soldTickets, returnedTickets`.
- **Methods:**
 - `searchTickets(dest): void` — delegates to DB search, prints via UI.
 - `blockTicket(ticketId): bool` — enforce AVAILABLE→BLOCKED.
 - `sellTicket(ticketId, p): bool` — enforce BLOCKED→SOLD, log sale.
 - `returnTicket(ticketId, travelDate): bool` — enforce SOLD→RETURNED, compute refund.
 - `generateReport(): void` — print totals with timestamp.

7) File Structure Mapping (.hpp / .cpp)

```
Types.hpp           // domain enums & structs
Employee.hpp       // base class
IClock.hpp         // clock interface
SystemClock.hpp    // clock implementation (header-only)
ConsoleUI.hpp/.cpp // UI helpers
RailwayConnectionDatabase.hpp/.cpp // repository
```

```

Cashier.hpp/.cpp          // application logic
main.cpp                  // wiring & scenario script

Build order example: Types.hpp → Employee.hpp → IClock.hpp &
SystemClock.hpp → RailwayConnectionDatabase.hpp/.cpp → ConsoleUI.hpp/.cpp
→ Cashier.hpp/.cpp → main.cpp.

```

8) Traceability Matrix (Requirements → Design → Code)

Req ID	Requirement	Design Element(s)	File(s) / API	Test/Usage in main.cpp
R1	Console-based UI that prints tickets and messages	UI Layer / ConsoleUI	showTicket, showMessage	Search, block, sell, return logs
R2	Cashier can search available tickets by destination	Cashier.searchTickets delegating to DB	RailwayConnectionDatabase.searchTickets("Tallinn")	
R3	Block only AVAILABLE tickets	Validation rule; Cashier.blockTicket	getTicketById, cashier.blockTicket(2), updateTicketStatus(..., BLOCKED)	
R4	Sell only BLOCKED tickets	Validation rule; Cashier.sellTicket	updateTicketStatus(pt, SOLD)	sell(3) fails; sell(2) succeeds
R5	Return only SOLD tickets with penalty policy	Validation rule; Cashier.returnTicket + time abstraction	IClock::now, return(2, updateTicketStatus(10, d) → RETURNED)	10% penalty
R6	Reporting with timestamp	Cashier.generateReport using IClock::now_str	ConsoleUI::showMessage printed	
R7	Data stored in-memory with CRUD-like ops	RailwayConnectionDatabase	addTicket, getTicketById, search, updateTicketStatus	Ticket setup + transitions